

Software Testing

Software Testing

- Software testing is a process that involves verifying that a software system works as expected and meets user needs.
- Testing is intended to discover program defects before it is put into use.
- Without proper testing, software may contain bugs that cause system failure, data loss or security vulnerabilities.

Benefits of Testing

- Improves software quality and reliability
- Early identification and fixing of defects
- Improves customer satisfaction
- Reduces maintenance costs
- Reduces development time

Disadvantages of Testing

- Time-consuming and adds to the project cost
- Can slow down the development process
- Not all defects can be found
- Difficult to fully test complex systems

Methods of Testing - Manual Testing

- Manual testing is a technique where testers check the software without using automated tools.
- It is done to ensure basic features of the software work properly.
- A tester carries out tests on the software by following a set of predefined test cases, compares actual results with expected outcome.

Manual Testing – FMS Example

A tester wants to verify whether the system correctly records daily income.

- Test the income entry feature

Steps

1. Open FMS
2. Navigate to the income sheet
3. Enter test data e.g.
Date: 3/13/2026
Amount: 1,000,000
4. Click save

Manual Testing – FMS Example

Expected Result

- The system should save the income record
- Total daily income should update to UGX1,000,000

Actual Result

- Tester checks record is saved correctly
- Correct amount appears in daily income report

• *If wrong amount is displayed, tester reports a defect*

Pros of Manual Testing

- Simple and easy to perform
- Less expensive compared to automated testing
- Does not require high level skills and specialized tools
- Useful for testing frequently changing features

Cons of Manual Testing

- Time-consuming for large systems
- Prone to human errors
- Slower than automated testing
- Not good for repetitive tests

Methods of Testing - Automated Testing

- Automated testing is a technique where testing activities are performed using specialized tools.
- The tester writes test cases (scripts) using suitable automation tools.
- The scripts interact with the system, enter inputs, run operations, compare results and report defects.
- Common automated testing tools: Selenium, JUnit, TestNG etc.

Automated Testing – FMS Example

A tester wants to verify whether the system correctly distributes income into categories.

- Salaries - 40%
- Operations - 20%
- Food - 30%
- Development - 5%
- Loans - 5%

Automated Testing– FMS Example

Automated Test Process

1. A test script automatically enters 100,000 as income
2. The script runs the distribution function
3. The script checks whether system produces correct results for each category e.g. Salaries = 40% * income
4. The tool automatically compares actual and expected results.

- *If results are incorrect, system reports a test failure i.e. FAIL*

Pros of Automated Testing

- Allows for execution of repetitive tests
- Increases amount of test coverage
- Tests are reliable
- Minimizes errors
- Suitable for testing large systems
- Faster execution of tests

Cons of Automated Testing

- Costly
- Requires skill to use automation tools

Types of Testing - Functional Testing

- Functional testing is a type of testing that verifies whether the system works according to the specified requirements.
- Tester checks inputs, processing and outputs
- Functional tests include unit testing, integration testing, system testing, acceptance testing, regression testing.
- FMS Example: Tester checks if system records daily income, distributes income into categories and generates daily income report.

Types of Testing – Non-Functional Testing

- Non-Functional testing is performed to verify quality attributes of the application.
- Non-functional testing checks the performance of the system in different situations, usability, security, reliability, compatibility, scalability etc.
- FMS Example: Tester checks how system responds when many finance staff are entering income simultaneously, whether system is easy to use etc.

Types of Testing – Regression Testing

- Regression testing is performed after changes are made to a system to ensure that existing features still work correctly and that changes did not introduce new defects.
- FMS Example: Suppose a new category “Transport (5%)” is added, after this change, tester checks that income distribution still works correctly with new percentages and daily income report is accurate.

Types of Testing – Smoke Testing

- Smoke testing checks whether the basic functions of a system work after a new build or update.
- If smoke test fails, other testing is not done until defects are fixed.
- FMS Example: Launch FMS, enter small income, check category distribution, generate quick daily report to see what it displays.

Types of Testing – Fuzz Testing

- Fuzz testing is where the system is given random, unexpected or invalid inputs to check if it crashes, behaves incorrectly or exposes security vulnerabilities.
- FMS Example: Enter extremely large number or a negative value as daily income. System should reject invalid inputs but should not crash or corrupt data.

Levels of Testing – Unit Testing

- Software testing is done in different levels from small parts to the whole system.
- Unit testing involves testing individual modules or functions of the software to ensure that they are working as expected.
- Helps identify bugs early in the development process before they become more difficult and expensive to fix.
- Ensures that changes to the code do not introduce new bugs.
- Tools for unit testing include: NUnit, JUnit, Jasmine etc.
- FMS Example: Test the income distribution function to verify that it correctly splits the income into salaries, operations, food, development and loans with the correct percentages.

Levels of Testing – Integration testing

- Integration testing whether different modules interact correctly.
- It is used to identify and resolve any issues that may occur when different units of the software are combined.
- It helps to improve the overall reliability and stability of the software.
- Tools: pytest, TestComplete
- FMS Example: Test the connection between monthly expense recording and balance calculation.

Levels of Testing – System Testing

- System testing evaluates the overall functionality and performance of a complete and fully integrated software.
- It tests if the system meets the specified requirements and if it is suitable for delivery to the end-users.
- It tests the entire product to easily detect the defects that cannot be identified during the unit testing and integration testing.
- Tools: Jira, Selenium
- FMS Example: Check if alerts work when category balances are negative or when an expense exceeds the available balance for a certain category.

Levels of Testing – User Acceptance Testing

- Acceptance testing is conducted to determine whether a system satisfies user needs, requirements, and business processes.
- It enables the users and customers to determine whether system is ready for real world use.
- This testing helps the project team to know further requirements from the users.
- Entire functionality of the product is tested.
- FMS Example: A bursar tests the system to see if it can track daily income from different sources, calculate expenses and generate a monthly financial statement as required.

Levels of Testing –User Acceptance Testing

- Alpha testing is type of UAT performed inside the development organization before the product is released to customers.
- FMS Example: **Checking login and security features**
- Beta testing is conducted before the final release of the software. System is given to real users to test in the real working environment.
- FMS Example: **User generates reports and gives feedback on system response**

Discussion

- A school FMS records daily income and expenses, distributes income in categories, tracks balances and generates reports. Giving suitable examples, discuss the following non-functional tests for the school FMS.
 1. Performance testing
 2. Load testing
 3. Stress testing
 4. Compatibility testing
 5. Reliability testing
 6. Security testing
 7. Usability testing
 8. Scalability testing
 9. Maintainability testing
 10. Recovery testing
 11. Availability testing
 12. Portability testing
 13. Interoperability testing
 14. Soak testing
- Etc.

Types of Testing– Static Testing

- Static testing is a type of testing where the software is checked without executing the system.
- Testers review requirements documents, system designs, source code and test cases.
- Static testing detects errors early in SDLC improving quality.
- However, runtime errors cannot be detected.

Types of Static Testing

- **Review:** This is the examination of documents, design diagrams, code or test cases to identify errors before the system is built or executed.
- Reviews improve quality and reduce cost of fixing errors later but cannot detect runtime errors.
- Reviews can be done based on how the review is conducted (**methods**) or what is being reviewed (**types**).

Reviews Methods

- **Informal review:** Does not define a formal process or roles.
 - FMS requirement: “The system distributes income into categories”
 - A reviewer reads requirement and notices percentages are missing
- **Walkthrough:** Informal inspection where author explains the document while others ask questions and give feedback.
 - Developer (Author) explains how income is distributed
 - Testers ask: what happens if categories change

Reviews Methods

- **Technical review:** Conducted by technical experts who focus on correctness and standards
 - Expert checks if FMS integrates correctly with Mobile Money Service system
- **Inspection:** Formal and structured with defined roles.
 - Team inspects the algorithm and finds salary percentage is incorrect (0.45 instead of 0.40)

Types of Reviews

- **Requirement review:** Process of examining the requirements document to ensure it is clear, complete, consistent and testable.
 - Detects missing requirements
 - Prevents errors before design and coding
- FMS requirement: **"The system distributes income into categories"**
- **Errors:**
 - No categories specified
 - No percentages specified
 - No explanation for updating categories
 - No rule for handling negative input
- Write corrected requirement
- The system distributes income into categories: Salaries (40%), Operations (30%), Food (20%), Development (5%) and Loans (5%). The percentage should not exceed 100%. If a category is updated, the percentages should be adjusted. If a user enters a negative income, the system should display a message requesting for positive values only.

Types of Reviews

- **Design review:** This checks the system architecture or design to ensure it correctly represents the requirements.
 - Verifies design solves the problem
 - Ensures structure is flexible
- Suppose design is as follows;
 - income → distribution → database → reports
- **Find errors and suggest improvement in the design**

Types of Reviews

- **Code review:** This involves examining the actual program code for errors.
 - Improves system quality
 - Ensures coding standards are followed
- Study the code
 - **salaries = income * 0.45**
 - **total = salaries + operations**

Types of Reviews

- **Test case review:** This involves checking that test cases are correct, complete, clear and cover all requirements.
 - Ensures test cases align with requirements
- FMS requirement: **"The system shall distribute income into categories: Salaries 40%, Operations 20%, Food 30%, Development 5%, Loans 5%"**
- Test case to be reviewed

Test case ID	Description	Input	Expected Result
TC01	Verify income distribution	1,000,000	Salaries = 450,000, Operations = 200,000, Food = 300,000, Development = 50,000, Loans = 50,000

Types of Testing– Dynamic Testing

- Dynamic testing is type of testing where the software is executed and the tester checks how it behaves.
- Tester enters data and compares actual with expected results.
- Types of dynamic testing include; functional and non-functional testing.
- State advantages and disadvantages of dynamic testing.

Testing Techniques/ Approaches

- Testing techniques describe how much knowledge the tester has about the system's internal structure.
- These include black box testing, white box testing and gray box testing .

Testing Techniques/ Approaches – Black Box Testing

- Testing the system without knowing the internal structure. Done by testers and users
- Black box testing focuses on inputs and outputs.
- Example
 - Input
 - Income = 1,000,000
 - Expected result
 - Salaries = 400,000
- *Tester is not concerned with how system calculates distribution*

Testing Techniques/ Approaches – Black Box Testing

- Black box techniques include;
 - *Equivalence partitioning*
 - *Boundary value analysis*
 - *Error guessing*
 - *Decision table testing*
 - *Cause-effect graphing*

Testing Techniques/ Approaches – White Box Testing

- Involves testing the system with full knowledge of the internal code and logic. Done by developers
- Focuses on code structure, logic paths, conditions and loops.
- Example
 - `if (income > 0) {`
 `salaries = income * 0.40`
 `}`
- *Tester checks if condition is correct and that all paths are executed*

Testing Techniques/ Approaches – Black Box Testing

- White box techniques include;
 - *Statement coverage*
 - *Branch coverage*
 - *Path testing*
 - *Condition coverage*
 - *Data flow analysis*
 - *Control flow analysis*

Testing Techniques/ Approaches – Gray Box Testing

- Combines black box testing and white box testing.
- Involves testing the system with partial knowledge of the system's internal implementation.
- Example
 - Tester knows percentages are stored in a database
 - Tester can change Salaries category to Allowances
 - Checks if system updates correctly



END